# LaTeX Author Guidelines for CVPR Proceedings

Anonymous CVPR submission

Paper ID *****

## Abstract

*[TODO: Abstract (150–200 words). Problem: floor-plan retrieval from incomplete queries (RPLAN). Method: frozen vision encoder + GNN on the room graph, combined by late fusion. Main result: fusion over the best single branch on the test split, read once (project status.md §53). Per-axis relevance only as descriptive analysis.]*

## 1. Introduction

A floor plan can be searched as an image or as a structure. Architects and designers often start from a layout that is only partially specified—a few rooms, an outline, a fragment of an existing plan—and look for complete plans that contain it. Retrieval in this setting must *complete* the query, not merely find plans that look like it. Two plans may look alike in a rendering while serving different functions, and two plans with the same rooms and connections may look different once rotated or resized.

The two natural representations of a plan fail in different ways when the query is incomplete. An image encoder sees the shape of what remains, including the shape of the missing part, but reads the room categories only through the colours of the rendering. A graph of rooms and adjacencies encodes the functional structure explicitly, but a partial graph is often the plausible *complete* graph of many other plans. Previous work on floor-plan similarity learns a similarity between complete plans, either from graphs [**?** **?** ] or, in general image retrieval, from global image descriptors [**?** ]; how these representations behave when part of the plan is missing, and whether they can be combined, has not been measured.

We study this question on RPLAN [**?** ], which provides both a rendered image and a structural record for the same plans. We build two retrieval branches on the same gallery of 67,405 plans: a vision branch with a frozen foundation encoder and whitening, chosen among 52 configurations over 8 encoders, and a graph branch whose GNN is trained to map a partial room graph onto its complete version. A late fusion concatenates the two descriptors with a single weight. We measure robustness as the ability to retrieve the source plan when rooms, a region or image patches are removed, at three levels of damage, and we make every choice on the validation split and read the test split once, with the expected outcomes written in advance.

On the test split, with rooms removed, late fusion raises the self-recovery score from 0.470 for the best single branch, the graph, to 0.642 ($+0.172$, 95% CI $[+0.162, +0.183]$), and the vision branch alone reaches 0.396. Two controls on the validation split show that the gain is not only an ensemble effect: fusing two trainings of the same graph encoder gains $+0.034$ over the better one, fusing two different vision encoders gains $+0.098$, and fusing image and graph gains $+0.174$. Figure 1 shows the mechanism on one query: usually one branch keeps the source near the top while the other demotes the plans that outrank it.

The path to these numbers also produced negative results that we report as such. Our first room-removal damage left the internal walls of the removed rooms in the image, so that each removed room remained visible as an empty outlined cell; this artifact inflated the measured robustness of the vision branch by a large margin. A projection head trained on frozen features to recover damaged plans improves on the damage it was trained on but loses on the other two, even after the artifact was fixed. Finally, relevance measures computed from the metadata are circular for both branches, so we use them only to describe what each system retrieves, never to select.

In summary, we make the following contributions:

- A protocol for floor-plan retrieval from incomplete queries: three damages that remove information at controlled levels, a self-recovery robustness metric fixed before the experiments, paired statistics, and a test split read once.
- An image branch and a graph branch evaluated on the same gallery and queries, and a late fusion that improves self-recovery under room removal by $+0.17$ over the best branch on the test split, with two controls that separate the ensemble effect from the complementary information.
- Negative results with their mechanism: a rendering arti-

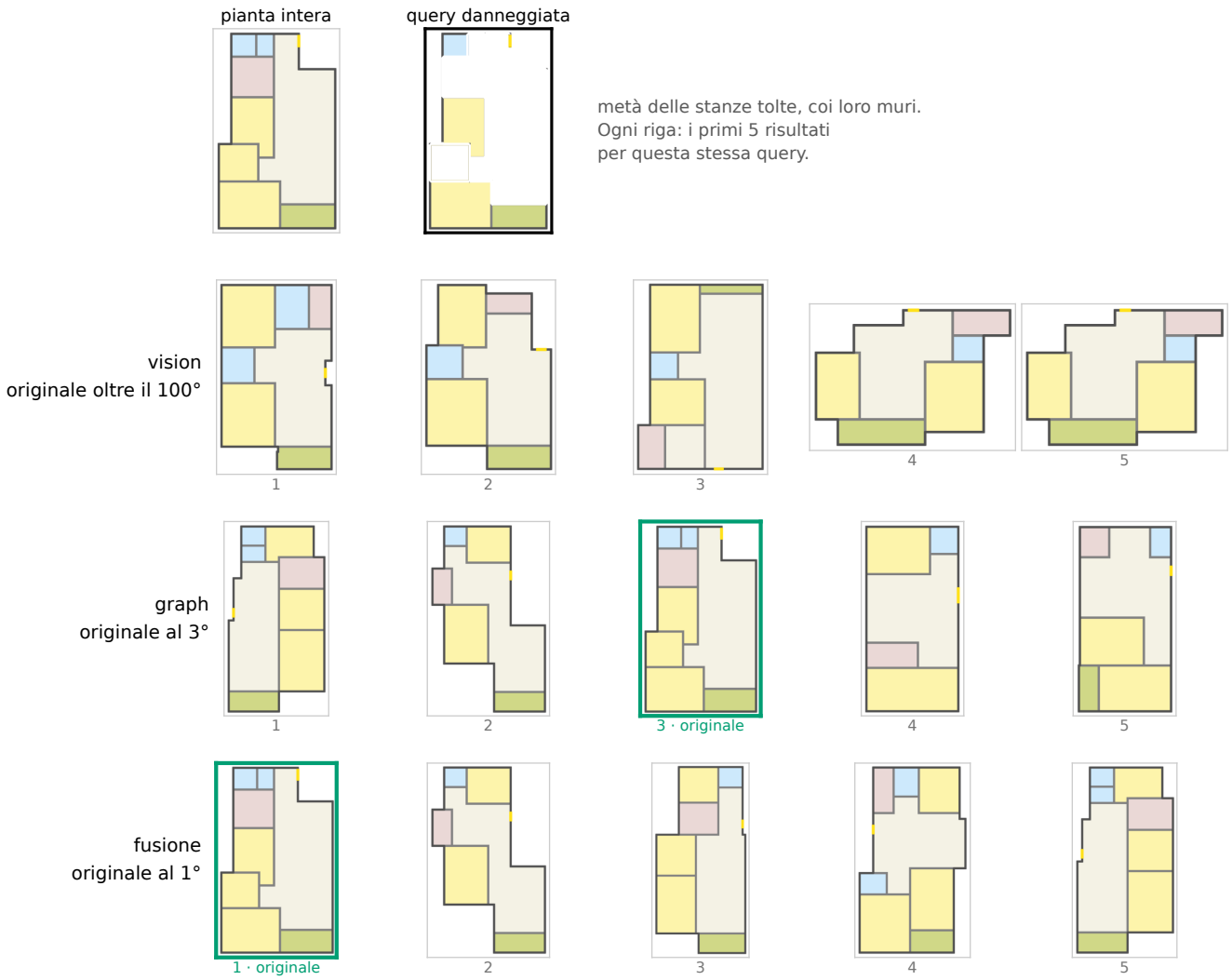



Figure 1. **Retrieving a plan from an incomplete query.** Top: a validation plan and the query obtained by removing half of its rooms together with their walls. Each row shows the top-5 results of one system, with the source plan framed where it appears. The vision branch does not find the source in its top 100 and the graph branch ranks it third; yet the vision branch scores the source above the two plans the graph ranks first, and late fusion ranks it first. The query is the first, in alphabetical order, of the 423 validation queries at this damage level in which fusion ranks the source first and neither branch does, so it illustrates the mechanism and not the average case (Sec. 4.3).

fact that inflated the measured robustness, and a trained projection head that learns the damage it sees rather than a general robustness.

## 2. Related Work

**Floor-plan datasets.** RPLAN [**?** ] provides more than 80,000 residential floor plans with room-level annotations, and is widely used for learning on floor plans [**? ? ?** ]. Its layouts are comparatively simple, with mostly rectangular rooms, a limitation that motivates more recent datasets such as ResPlan [**?** ], which offers 17,000 vector plans with room connectivity; CubiCasa5K [**?** ] instead targets the parsing of floor-plan images, with 5,000 plans densely annotated into more than 80 object categories. We use RPLAN because, for the same plans, it provides both a rendered image and a structural record, the two inputs our branches compare.

**Floor plans as graphs.** Representing a plan as a graph of rooms connected by adjacencies is common in floor-plan generation. Graph2Plan [**?** ] generates a floor plan from a layout graph and a building boundary, and obtains its input graphs by retrieving, from RPLAN, plans that match the boundary and the user's room constraints. Our graph branch uses the same kind of room graph, built directly from the RPLAN records, as the input of a retrieval encoder rather than of a generator.

**Floor-plan retrieval and similarity.** Retrieving floor plans requires a notion of similarity that is not purely visual. LayoutGMN [**?** ] learns a structural similarity between layouts with an attention-based graph matching network trained on triplets whose labels come from pixel-wise IoU, and evaluates retrieval on RPLAN plans and user-interface designs. LayoutGKN [**?** ] postpones the cross-graph interactions to a differentiable graph kernel on the final node embeddings, which makes inference much faster than graph matching, and trains on RPLAN with a similarity based on graph edit distance, which the authors find to correlate with human judgement far better than IoU. Both methods learn a similarity between complete plans from a pairwise target. Neither addresses incomplete queries, which is the setting we study, and neither compares an image-based and a graph-based representation of the same plans or combines them.

**Visual foundation encoders.** Frozen general-purpose encoders are a natural baseline for retrieval on a new image domain. We consider self-supervised encoders trained on curated data (DINOv2 [**?** ], DINOv3 [**?** ]), the joint-embedding predictive I-JEPA [**?** ], which predicts the representation of masked target blocks from a context block and is therefore conceptually close to incomplete inputs, vision–language encoders (SigLIP 2 [**?** ], TIPSv2 [**?** ]), the agglomerative AM-RADIO [**?** ], distilled from several teachers, and the Perception Encoder [**?** ], whose spatially aligned variant is tuned for dense features. For global descriptors we follow standard practice in instance retrieval: generalised-mean pooling [**?** ], PCA whitening of the descriptors [**?** ] and exact nearest-neighbour search with FAISS [**?** ].

**Graph neural networks and contrastive learning.** Graph encoders aggregate information from the neighbourhood of each node: GCN [**?** ] with a first-order approximation of spectral convolutions, GraphSAGE [**?** ] with learned aggregators that generalise inductively to unseen graphs, and GATv2 [**?** ] with a dynamic attention that fixes the static attention of the original graph attention network. We train the graph encoder with the InfoNCE objective [**?** ], contrasting positive pairs against in-batch negatives; our positives pair a partial plan with its complete version, so that the learned embedding targets the incomplete-query task directly.

# 3. Method

Let $\mathcal{G}$ be a gallery of floor plans, each available both as a rendered image $I_p$ and as a structural record $S_p$ (room types, room boxes, typed adjacencies). A query is an *incomplete* version $\tilde{q}$ of a plan $q \in \mathcal{G}$, from which part of the content has been removed (Sec. 4.1); the task is to rank the gallery so that the source plan $q$ appears as high as possible. Two encoders map a plan to a unit vector: a vision encoder $v(\cdot) \in \mathbb{R}^{768}$ that reads the image and a graph encoder $g(\cdot) \in \mathbb{R}^{128}$ that reads the room graph. Both branches index the same gallery and receive the same damaged queries, so the two branches and their fusion are compared query by query (Fig. 2).

## 3.1. Late fusion

The fused descriptor is the weighted concatenation of the two L2-normalised vectors,

$$z_p = \left[\sqrt{\alpha}\, v_p \,;\, \sqrt{1-\alpha}\, g_p\right], \qquad \langle z_q, z_p\rangle = \alpha\, \langle v_q, v_p\rangle + (1-\alpha)\, \langle g_q, g_p\rangle, \tag{1}$$

so that $\|z_p\| = 1$ and a single exact inner-product search over the 896-d vectors ranks by a convex combination of the two cosine similarities. The endpoints recover the single branches exactly ($\alpha = 1$ vision, $\alpha = 0$ graph), which we use as a correctness check of the fused pipeline. For a damaged query, both query vectors are recomputed from the *same* removed rooms, while gallery vectors are those of the complete plans.

The weight is chosen on the validation split over $\alpha \in \{0, 0.1, \ldots, 1\}$ with a rule fixed before the runs: the criterion is the robustness to room removal (Sec. 4.1), the only damage both branches can receive; the candidates whose paired difference from the best has a 95% bootstrap interval containing zero are considered tied, and the central one of the tied set is selected. This gives $\alpha^* = 0.6$, which is then kept fixed on the test split.

## 3.2. Graph branch

**Room graph.** Each room of the record becomes a node, built directly from the RPLAN metadata [**?** ] rather than by parsing the image. Node features concatenate a one-hot encoding of the 13 room types with six geometric features of the room box on the $256 \times 256$ RPLAN grid: centre $(c_x, c_y)$, width, height, area and aspect ratio. The aspect ratio is clipped at its 99th percentile and the geometric features are standardised with training-split statistics. Adjacencies become undirected edges whose features are a one-hot encoding of the 10 RPLAN relation types.

**Encoder.** The graph encoder stacks two GATv2 layers [**?** ] with 4 attention heads (averaged), hidden size 128 and edge features in the attention, with a ReLU between layers. Node states are sum-pooled and concatenated with the sum-pooled input features; on the one-hot part the latter is exactly the room-type histogram, which therefore reaches the output without depending on the learned weights. A linear layer projects the result to 128 dimensions, followed by L2 normalisation.

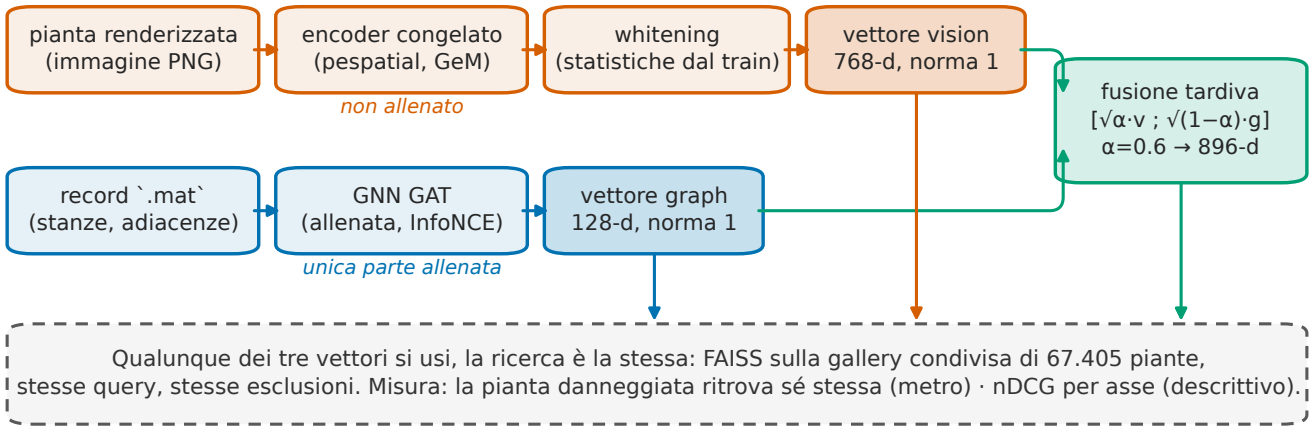



Figure 2. **Pipeline overview.** Every floor plan is available both as a rendered image and as a structural record. The vision branch embeds the image with a frozen encoder followed by whitening; the graph branch embeds the room graph with a GNN trained for incompleteness. Late fusion concatenates the two unit vectors with weights $\sqrt{\alpha}$ and $\sqrt{1-\alpha}$ ($\alpha = 0.6$), so that one inner product retrieves with both. All three systems search the same gallery with the same queries and are scored with the same measure.

**Training for incompleteness.** The encoder is trained with the symmetric InfoNCE loss [**?** ] with in-batch negatives, temperature $\tau = 0.3$, batch size 256 and AdamW (learning rate $10^{-3}$, weight decay $10^{-5}$), on the training split only. Positive pairs are *asymmetric*: one view is the complete plan, randomly reflected ($p = 0.5$) or rotated by a multiple of 90° ($p = 0.5$); the other view is the same plan with $\mathrm{round}(f\,n)$ of its $n$ rooms removed, $f \sim \mathcal{U}[0.25, 0.75]$, keeping the subgraph induced by the remaining rooms and at least one room. The pair thus asks the encoder to map a partial plan onto its complete version, which is the retrieval task of Sec. 4.1. The checkpoint is not selected on the training loss: after each epoch we measure the robustness to room removal on 2,000 validation queries disjoint from the evaluation queries, and keep the best epoch within a budget of 300.

### 3.3. Vision branch

**Encoder.** The vision branch uses a frozen Perception Encoder [**?** ], in its spatially aligned ViT-B/16 variant (PE-spatial), without any training. The rendered plan is resized so that its longer side is 224 pixels, keeping the aspect ratio, and padded to a square with the white background of the renderings. The global descriptor is the generalised-mean (GeM) pooling [**?** ] of the last-layer patch tokens with exponent $p = 3$, followed by L2 normalisation.

**Whitening and indexing.** We apply PCA whitening [**?** ], whose effect is measured in Sec. 4.3.1: the mean $\mu$ and the covariance of the descriptors are estimated on the training-split plans only, and every descriptor $x$ is mapped to $W^\top(x-\mu)$ with $W = U\Lambda^{-1/2}$, keeping all 768 components, and re-normalised. Queries undergo the same transformation. Gallery vectors are stored in an exact inner-product FAISS index [**?** ], so that the score is the cosine similarity.

**Choice of the configuration.** Encoder, pooling, input resolution and whitening are not fixed a priori: PE-spatial with GeM pooling and whitening is the most robust of 52 frozen configurations over 8 encoders [**? ? ? ? ? ? ? ?** ] on the validation split (Sec. 4.3.1).

## 4. Experiments and Results

### 4.1. Metrics

**Incomplete queries.** A query is a validation or test plan from which a fraction $f \in \{0.25, 0.5, 0.75\}$ of the content is removed with one of three damages, while the gallery keeps the complete plans. *Room removal* deletes $\mathrm{round}(f\,n)$ of the $n$ rooms, drawn uniformly with a per-query seed so that both branches remove the same rooms. In the image, the removed rooms are filled with the background colour and their walls are erased, whatever their shade, so that the query loses the room type and its partitions and keeps only the shape of the hole; in the graph, only the subgraph induced by the remaining rooms is kept. Since rooms differ in size, the removed fraction of the plan area is on average 0.29, 0.52 and 0.71 at the three levels. *Crop* blanks one axis-aligned rectangle of random sides and position, accepted when it covers a fraction $f \pm 0.05$ of the plan pixels. *Patch* blanks whole ViT patches of the resized input, in random order, until a fraction $f$ of the plan is covered, so that entire tokens are removed. Crop and patch act on pixels and are

applied to the vision branch only.

**Robustness metric.** We measure *self-recovery*: the gallery still contains the intact source plan, and a query scores the reciprocal rank $\mathrm{rr}(q, f)$ of its source among the top 100 results, and 0 beyond. For a damage $d$, the per-query robustness is the mean over the three levels, $\mathrm{AUC}_d(q) = \frac{1}{3}\sum_f \mathrm{rr}_d(q, f)$, and for the vision branch the overall score averages the three damages, $R(q) = \frac{1}{3}\sum_d \mathrm{AUC}_d(q)$, so that no configuration can win by exploiting a single damage. Branches and fusion are compared on room removal, the damage they share. We report means over queries; every comparison is paired over the same queries and removed rooms, with 95% percentile bootstrap intervals over queries ($B = 10{,}000$). The level $f = 0$ is excluded: about 3% of the RPLAN images have a pixel-identical twin, which caps the undamaged self-recovery at about 0.97 for every system, a ceiling of the data rather than of the models.

**Per-axis relevance (descriptive).** To describe what a system retrieves beyond the source plan, we score the ranking of the complete plan against three relevance axes computed from the metadata, each in $[0, 1]$. *Composition* is the weighted Jaccard similarity between the 13-bin room-type histograms; *topology* is the weighted Jaccard similarity between the 91-bin histograms of adjacent room-type pairs, which is invariant to room order; *geometry* averages the similarity of footprint area ($1 - |a_q - a_p|$ on the normalised grid), of footprint aspect ratio (ratio of the smaller to the larger) and of the per-type area distribution (one minus half the $\ell_1$ distance). We report nDCG@10 with these similarities as graded gains, with the ideal ranking computed over the whole gallery and the query itself excluded. These scores are *descriptive* and never used for selection, because they are circular for both branches: the graph encoder receives the room types and adjacencies that composition and topology measure, the footprint is recoverable from the room boxes, and the renderings encode the room types as colours, albeit with 13 types collapsed into 6 colours. We use them to state the cost of a choice on complete plans, not to rank systems.

## 4.2. Dataset and experimental details

**Dataset.** We use RPLAN [**?** ], a large collection of residential floor plans that provides, for each plan, a rendered colour image (1167 × 875 pixels, rooms coloured by category) and a structural record with 13 room types, room boxes and typed room adjacencies. The gallery contains the 67,405 plans that have both an image and a record (48 of the 67,453 images have no record and are excluded), and it is shared by the two branches, which index exactly the same plans in the same order. We follow the official split, which assigns 47,126 gallery plans to training, 10,152 to validation and 10,127 to test. The graph encoder is trained and the whitening is estimated on training plans only, while the gallery always contains all plans, as in a search engine.

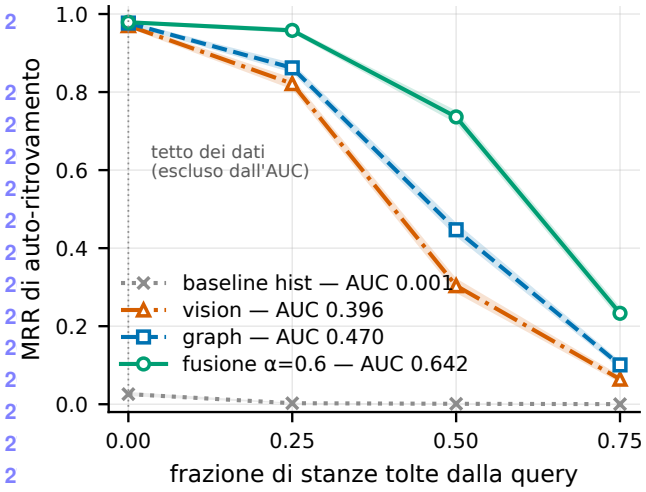



Figure 3. **[TODO: Test split: self-recovery vs. fraction of removed rooms for hist, vision, graph and fusion. Source: f_damage_test.sources.txt.]**

**Protocol.** Every choice (vision configuration, graph checkpoint, fusion weight) is made on 2,000 validation queries sampled with a fixed seed. The test split is used once, after all choices were frozen and the expected outcomes written down, with 2,000 test queries sampled in the same way; no number reported on the test split was used to select anything. Queries remain in the gallery, since retrieving them is the self-recovery task.

## 4.3. Results

### 4.3.1. Ablation

**[TODO: Ablation table (mean ± std, pending D1): vision configurations under the three damages; graph; fusion weight $\alpha$ and the two controls. Main test result (T3, project status.md §53).]**

**Negative results.** **[TODO: Wall artifact of the historical room masking (project status.md §35–§37); the trained projection head does not transfer to other damages (§37, §48); the lost-neighbours marker ties (§47).]**

### 4.3.2. Qualitative results

**[TODO: Qualitative examples by model and on the dataset (Fig. 1).]**

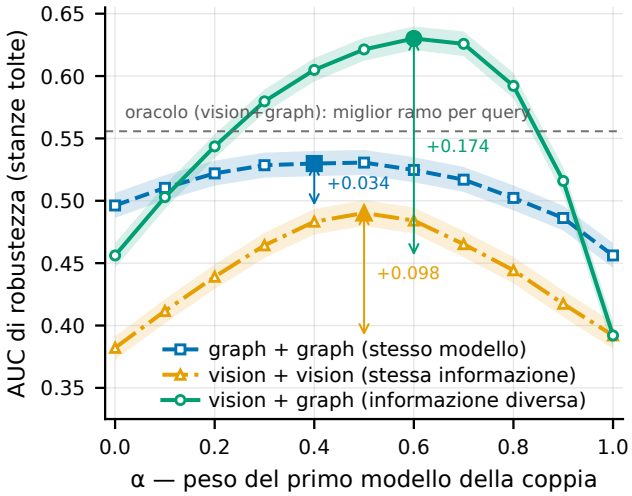



Figure 4. **[TODO: Valid split: robustness AUC vs. $\alpha$ for the three fused pairs. Source: f_alpha_valid.sources.txt.]**

## 5. Conclusions

**[TODO: Summary; limitations (PAPER.md §8: RPLAN only, synthetic damages, proxy relevance, fusion tied to this graph checkpoint); future work (PAPER.md §9).]**

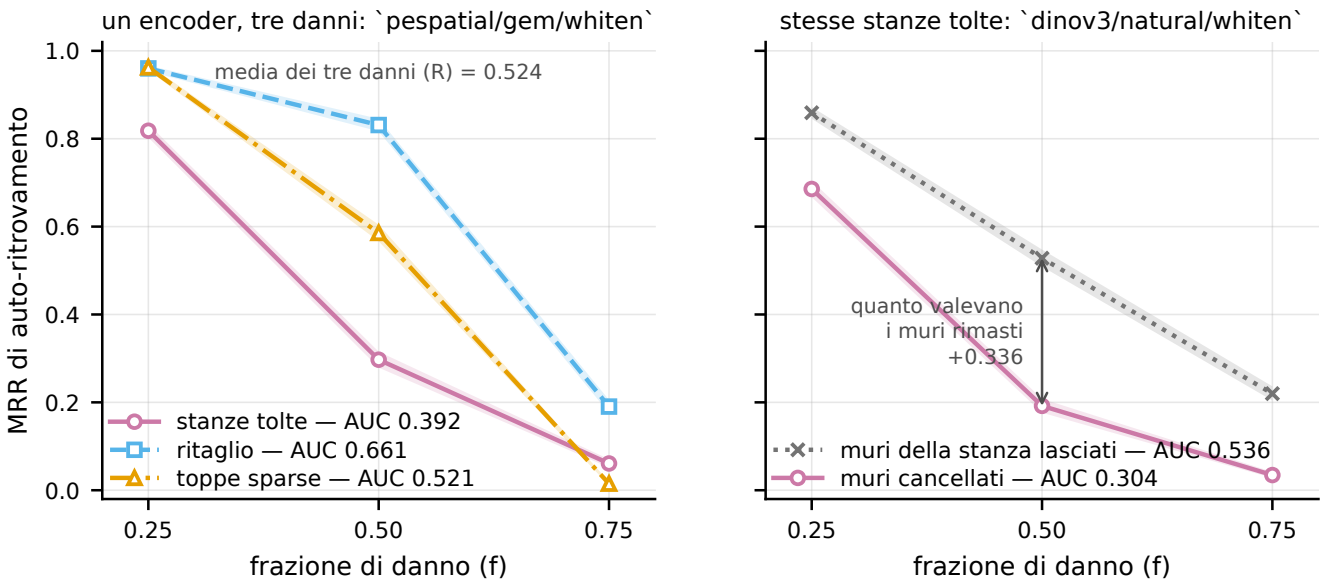



Figure 5. **[TODO: Left: final vision config under the three damages. Right: the wall artifact, same removed rooms with walls kept or erased. Source: f_damage_kinds_valid.sources.txt.]**